

Advanced Data Models SoSe 2026

Document Stores

Lars Wieck & Luca Raichle

April 2026

Agenda

01	Motivation	10 min
02	Konzeptionelles Modell	15 min
03	Datenstruktur	15 min
04	Abfragemodell	15 min
05	Architekturüberblick	15 min
06	Vergleich mit rel. Datenbanken	10 min
07	Praktische Demo	30 min
08	Challenge	20 min
09	Kahoot Quiz	10 min
10	Diskussion	10 min

01

Motivation

10 Minuten

Schritt 1: Homogene Entitäten

Ein Online-Händler verkauft ausschließlich Bücher - alle Produkte teilen dieselben Attribute.

Tabelle: PRODUKTE

ISBN	Titel	Autor	Verlag	Seiten	Preis
978-0134685991	Effective Java	Bloch	Addison-Wesley	412	39,90 €
978-0132350884	Clean Code	Martin	Pearson	464	34,90 €
978-0135957059	Pragmatic Programmer	Hunt / Thomas	Addison-Wesley	352	29,90 €
978-0135957002	Advanced Data Models Esslingen	Seyler	Esslingen-Verlag	222	24,90 €

**Sauberes Schema · alle Felder gefüllt · keine Redundanz => relationales Modell ist optimal
...aber die Realität bleibt selten so einfach**

Schritt 2: Erste Heterogenität

Das Sortiment wird um Musik-Alben erweitert. Schema-Erweiterung per ALTER TABLE → neue Spalten für Musik-spezifische Attribute.

Tabelle: PRODUKTE (mit neuen Spalten für Musik)

ISBN	Titel	Autor	Seiten	Preis	Künstler	Album	Tracks	Label
978-013...	Effective Java	Bloch	412	39,90 €	NULL	NULL	NULL	NULL
978-013...	Clean Code	Martin	464	34,90 €	NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL	12,99 €	Beatles	Abbey Road	17	Apple
NULL	NULL	NULL	NULL	14,99 €	Pink Floyd	The Wall	26	Harvest
NULL	NULL	NULL	NULL	11,99 €	Daft Punk	Discovery	14	Virgin

Erste NULL-Werte: Attribute, die für manche Entitäten nicht existieren
SQL-Standard: NULL bedeutet „unbekannter Wert“, in diesem Fall aber „nicht definiert“
=>Fundamentale Verletzung

Schritt 3: Sparse Tabelle

Mit jeder weiteren Kategorie wächst die Tabelle in der Breite - jede Zeile nutzt aber nur einen Bruchteil der Spalten.

Tabelle: PRODUKTE — 12 Spalten, jede Zeile füllt nur 2–3 davon

ISBN	Autor	Seiten	Künstler	Album	Tracks	Regisseur	FSK	Watt	Energie- klasse	Größe	Material
978-013...	Bloch	412	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
978-013...	Martin	464	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
NULL	NULL	NULL	Beatles	Abbey Road	17	NULL	NULL	NULL	NULL	NULL	NULL
NULL	NULL	NULL	Pink Floyd	The Wall	26	NULL	NULL	NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	Nolan	12	NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	1.200	A++	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	M / Rot	Baumwolle

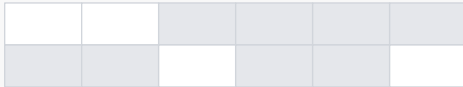
Speicher-Overhead · NULL-Mehrdeutigkeit · Schema-Migration bei jeder neuen Kategorie

Wie löst man das?

Diese Situation ist bekannt und es gibt daher eine Art Standardrezept zur Behebung.

Attribute, die nur für eine Teilmenge der Entitäten gelten,
gehören in eine eigene Tabelle.

1 breite Tabelle



Spezialisierung

1 Basis + N Subtyp-Tabellen



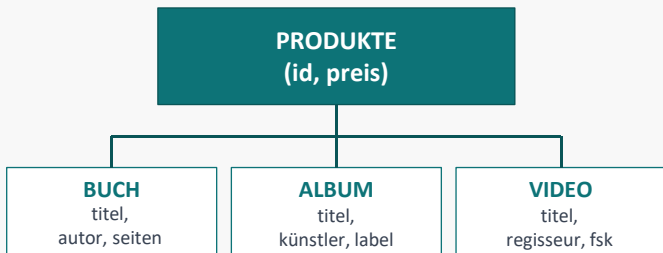
Schritt 4: Workaround Subtyp-Tabellen



Eine ganz alltägliche Anfrage: (Wer ist der Urheber des Produkts)

Welche Produkte sind von „Beatles“?

Schema



*„Urheber“ heißt pro Typ anders: autor /
künstler / regisseur*

Notwendige SQL:

```
SELECT
  p.id, p.preis, b.titel, a.titel, v.titel
FROM produkte p
LEFT JOIN buch b ON b.produkt_id = p.id
LEFT JOIN album a ON a.produkt_id = p.id
LEFT JOIN video v ON v.produkt_id = p.id
WHERE
  b.autor      ILIKE '%Beatles%'
OR a.künstler  ILIKE '%Beatles%'
OR v.regisseur ILIKE '%Beatles%';
```

Pro neuer Kategorie: ein neuer LEFT JOIN + ein neues OR + ein neuer Spaltenname

Das ist keine Theorie.

Ein sehr ähnliches Problem trat auch bei Amazon auf – von Relationalen schrittweise zu flexibleren, verteilten Datenmodellen.



Jede Produkterweiterung stellte das Datenmodell vor dieselben Probleme:

Relationales Datenmodell nicht mehr tragbar

Heterogene, evolutionäre Entitäten brauchen ein Datenmodell, das das nativ unterstützt.

→ Document Stores

Schritt zurück: Warum NoSQL?

Wir haben einen Treiber gesehen. NoSQL als Bewegung hatte einen zweiten.

1

Flexibilität

- Heterogene, sich entwickelnde Entitäten - wie wir gerade gezeigt haben
- Festes Schema vor dem Speichern vs. dynamische Datenmodelle in der Realität

2

Skalierung

- Web-Plattformen mit Millionen Nutzern produzieren enorme Datenmengen
- RDBMS skalieren primär vertikal (mehr Zeilen) - stärkere Maschine/Festplatte
- Horizontal über viele Server: dafür braucht es ein anderes Datenmodell (JOINS zu langsam)

Ende 2000er: NoSQL-Bewegung als Antwort auf beide Treiber gleichzeitig

Warum Document Stores im Speziellen?

NoSQL ist eine Familie. Vier große Untertypen, jeder für ein anderes Problem.



Key-Value

einfache Schlüssel-Wert-Paare

z. B. Redis, Memcached

Document

schema-flexible
JSON-Dokumente

z. B. MongoDB, Couchbase

Wide-Column

tabellenartig mit
dynamischen Spalten

z. B. Cassandra, HBase

Graph

Beziehungen erster Klasse

z. B. Neo4j, Neptune

Document Store

Speichert Daten als strukturierte, JSON-ähnliche Dokumente - verschachtelbar, schema-flexibel und nah an der Objektstruktur der Anwendung.

→ Wie das konkret aussieht, behandeln wir im nächsten Kapitel.

Wo werden Document Stores eingesetzt?

Überall, wo Daten viele Felder haben, oft variabel und tief geschachtelt sind.



E-Commerce

Produkte mit kategorie-spezifischen Eigenschaften - wie unser Bücher-/Musik-/Video-Beispiel. Amazon (DynamoDB), Otto (MongoDB).



Content Management

Artikel, Blog-Posts, Medien-Inhalte mit variablen Feldern: Tags, eingebettete Medien, Kommentar-Bäume.



User Profiles

Optionale Attribute, die mit der Zeit dazukommen - Präferenzen, Verknüpfungen mit Diensten, Verhaltens-Metadaten.



KI & ML-Pipelines

Trainingsdaten, Features, Modell-Metadaten und Logs - meist semi-strukturiert, oft pro Run/pro Modell unterschiedlich.

02

Konzeptionelles Modell

15 Minuten

Key-Value als Fundament

Wir brauchen ein flexibles Datenmodell. Bauen wir es auf - vom kleinsten Baustein her.

Das Modell



Der Wert ist für die DB ein undurchsichtiger Block, sie kann seinen Inhalt nicht interpretieren.

Was das kann, und was nicht

- ✓ **Blitzschnell & horizontal skalierbar**
Caches, Sessions - Redis, Memcached, DynamoDB
- ✓ **Minimales API**
get(key) und put(key, val) - mehr nicht.
- ✗ **Keine Suche im Wert da Blackbox**
„Alle Produkte von Beatles“: unmöglich direkt, nur mit jedem Wert laden
- ✗ **Keine Abfrage nach Feldern**
Wert bleibt ein undurchsichtiger Block.

Das reicht für Session-Caches. Nicht für unseren Produktkatalog.

Fortschritt: Dem Wert eine Struktur geben

Was passiert, wenn die DB den Inhalt des Werts interpretieren kann?

dieselbe Box, aber jetzt durchsichtig

"user:42"

+ Struktur



```
{
  "_id": "u_42",
  "name": "Alice",
  "tier": "gold"
}
```

Gleicher Container - aber die DB sieht jetzt hinein und kann nach Feldern abfragen.

LÖSUNG FÜR DAS PROBLEM AUS KAPITEL 1

Jedes Produkt wird sein eigenes Dokument.

Bücher, Alben, Videos - jedes mit eigenen Feldern, alle in einer Collection. Ohne NULLs. Ohne JOINS.

Was ist ein Dokument?

Eine strukturierte Dateneinheit, die ein ganzes Anwendungsobjekt repräsentiert (Schritt für Schritt aufgebaut/erweiterbar)

```
{
  "_id":      "p_007",
  "kategorie": "buch",
  "titel":    "Effective Java",
  "preis":    39.90,

  "autor": {
    "name": "Joshua Bloch",
    "land": "USA"
  },

  "tags": ["java", "patterns"],

  "lager": [
    { "ort": "Berlin", "anzahl": 12 },
    { "ort": "Hamburg", "anzahl": 4 }
  ]
}
```

Bestandteile eines Dokuments

■ **_id**

Eindeutiger Identifikator innerhalb der Collection.

■ **Felder**

Name + Wert (String, Number, Boolean, ...).

■ **Verschachtelte Objekte**

Hierarchien wie z. B. autor.name.

■ **Arrays**

Listen primitiver Werte oder ganzer Sub-Objekte.

Ein Dokument repräsentiert ein ganzes Objekt - kein verteiltes Schema über mehrere Tabellen
„Locality of Reference“: Alles was zusammengehört liegt physisch auf dem Speicher beieinander

Was kann ein Dokument repräsentieren?

Ein Dokument fasst ein vollständiges Anwendungsobjekt - egal aus welcher Domäne.



Produkt · E-Commerce

```
{ "_id": "p_007", "kategorie": "buch",  
  "titel": "Effective Java", "preis": 39.90,  
  "lager": [{"ort": "Berlin", "anzahl": 12}] }
```



Nutzerprofil · Identity

```
{ "_id": "u_42", "name": "Alice",  
  "email": "alice@test.de", "tier": "gold",  
  "praeferenzen": {"sprache": "de"} }
```



Bestellung · Order

```
{ "_id": "b_889", "kunde_id": "u_42",  
  "items": [{"id": "p_007", "stk": 1}],  
  "summe": 39.90, "status": "versendet" }
```



Blog-Post · CMS

```
{ "_id": "a_215", "titel": "Doc Stores 101",  
  "autor": "Alice", "tags": ["nosql", "db"],  
  "kommentare": [{"von": "Peter", "text": "..."}] }
```

Prinzip: 1 Anwendungsobjekt = 1 Dokument - egal ob Produkt, Nutzer, Bestellung oder Artikel
=>Die Datenbankstruktur passt sich der Anwendungslogik an, nicht umgekehrt

Tabelle vs. Collection



Worin werden die Dokumente dann gespeichert?

Was im relationalen Modell die Tabelle ist, ist im Document Store die Collection - mit einem entscheidenden Unterschied.

Tabelle (relational)

Festes Schema, alle Zeilen identisch strukturiert.

id	kategorie	titel	preis
007	buch	Effective Java	39.90
012	buch	Clean Code	34.90
018	buch	Java Concur.	29.90

Jede Zeile muss das Schema erfüllen, also
Constraints auf Datenbankebene

Collection (Document Store)

Schema-flexibel: Dokumente unterschiedlich (z.B. Collection PRODUKTE)

```
{ _id:007, kategorie:"buch", titel:"Effective Java", autor:"Bloch", isbn:"..." }
```

```
{ _id:012, kategorie:"album", titel:"Abbey Road",  
künstler:"Beatles", tracks:17 }
```

```
{ _id:018, kategorie:"video", titel:"Inception",  
fsk:12, regisseur:"Nolan" }
```

Datenbank erzwingt keine einheitliche Struktur,
leicht neue Produkte/Kategorien einzufügen
ABER: Schema-Flexibilität != Schema-Losigkeit
(Schema Validation möglich)

Genau das löst unser Heterogenitäts-Problem aus Kapitel 1 - ohne NULLs, ohne JOINS.
=>Flexibilität wo sie gebraucht wird, Constraints wo sie sinnvoll sind.

Konzepte übersetzt: Relationale DB ↔ Document Store

Die meisten Begriffe haben ein direktes Pendant. Nur die Struktur dahinter ist anders.

RELATIONAL		DOCUMENT STORE
Tabelle <i>Behälter für Zeilen</i>	→	Collection <i>Behälter für Dokumente</i>
Zeile <i>feste Spalten, einheitlich</i>	→	Dokument <i>eigene Felder pro Eintrag</i>
Spalte <i>ein Datentyp pro Spalte</i>	→	Feld <i>beliebiger Typ, auch verschachtelt</i>
Primärschlüssel <i>eindeutige Zeilen-ID</i>	→	_id <i>eindeutige Dokument-ID</i>
JOIN <i>Tabellen über Fremdschlüssel</i>	→	embedded oder referenziert
Festes Schema <i>alle Zeilen identisch</i>	→	Schema-flexibel <i>pro Dokument unterschiedlich</i>

Beziehungen modellieren - zwei Wege

Dieselbe Bestellung - zwei verschiedene Dokument-Strukturen

Embedded

Items leben im Bestell-Dokument

```
{
  "_id": "b_889",
  "kunde": "u_42",
  "items": [
    { "id": "p_007", "preis": 39.90 },
    { "id": "p_012", "preis": 34.90 }
  ],
  "summe": 74.80
}
```

Ein Read holt die ganze Bestellung.

Referenced

*Items leben in eigener Collection, Dokument speichert
nur die Referenz*

```
{
  "_id": "b_889",
  "kunde": "u_42",
  "item_ids": [
    "p_007",
    "p_012"
  ],
  "summe": 74.80
}
```

Items werden bei Bedarf nachgeladen (mehrere Reads)

Beide Strategien sind valide. Wann was verwendet werden soll ist eine grundlegende Design-Entscheidung. Gleich dazu mehr.

03

Datenstruktur

15 Minuten

Wie werden Dokumente gespeichert?

Für den Menschen sieht ein Dokument wie JSON aus. Intern gespeichert & verarbeitet wird meist mit BSON

JSON (logisches Modell)

```
{
  "_id": "b_889",
  "kunde": "u_42",
  "items": [
    { "id": "p_007", "preis": 39.90 },
    { "id": "p_012", "preis": 34.90 }
  ],
  "summe": 74.80
}
```

lesbar, textbasiert, gut für Menschen & APIs

BSON (technische Repräsentation)

- binäres Format statt Text
- zusätzliche Datentypen, z. B. ObjectId, Date, Binary, Decimal128
- effizient für Parsing und interne Verarbeitung
- nicht menschenlesbar, sondern für die Datenbank optimiert

Schema Flexibilität & Validierung

Schema Flexibilität

Dokumente in derselben Collection dürfen sich unterscheiden

```
{ _id:007, kategorie:"buch", titel:"Effective Java", autor:"Bloch", isbn:"..." }
```

```
{ _id:012, kategorie:"album", titel:"Abbey Road",  
künstler:"Beatles", tracks:17 }
```

```
{ _id:018, kategorie:"video", titel:"Inception",regisseur:"Nolan",  
fsk:12 }
```

Schema Validierung

Flexibilität heißt nicht Beliebigkeit

```
{  
  "$jsonSchema": {  
    "required": ["title", "price"],  
    "properties": {  
      "title": { "bsonType": "string" },  
      "price": { "bsonType": "number" }  
    }  
  }  
}
```

Validierungsregeln können Pflichtfelder und Datentypen absichern

Document Stores sind nicht schemafrei, sondern schema-flexibel. Validation liefert Guardrails.

Embedding Beispiel: Website mit Blogposts

Artikel-Dokument

```
{
  "_id": "article_1001",
  "title": "Why Document Stores Matter",
  "slug": "why-document-stores-matter",
  "content": "Document stores are useful when...",
  "status": "published",
  "publishedAt": "2026-04-23T10:00:00Z",
  "tags": ["nosql", "document-store", "mongodb"],
  "category": {
    "name": "Databases",
    "slug": "databases"
  },
  "author": {
    "_id": "author_42",
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "bio": "Tech writer."
  },
  "comments": [
    {
      "_id": "comment_9001",
      "userName": "Bob",
      "text": "Great introduction!",
      "createdAt": "2026-04-23T12:15:00Z"
    },
    ...
  ]
}
```

Nested Kategorie-Dokument

Nested Autor-Dokument

Array von nested Kommentar-Dokumenten

Embedding Beispiel: Website mit Blogposts

```
{
  "_id": "article_1001",
  "title": "Why Document Stores Matter",
  "slug": "why-document-stores-matter",
  "content": "Document stores are useful when...",
  "status": "published",
  "publishedAt": "2026-04-23T10:00:00Z",
  "tags": ["nosql", "document-store", "mongodb"],
  "category": {
    "name": "Databases",
    "slug": "databases"
  },
  "author": {
    "_id": "author_42",
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "bio": "Tech writer."
  },
  "comments": [
    {
      "_id": "comment_9001",
      "userName": "Bob",
      "text": "Great introduction!",
      "createdAt": "2026-04-23T12:15:00Z"
    },
    ...
  ]
}
```

PROS

- Alle Artikelinformationen in einer Query (keine JOIN-Operationen)
- Zusammengehörige Daten können in einer atomaren Dokument-Operation gelesen und geändert werden

CONS

- Dokumenten Größenlimit (16-MB bei MongoDB)
→ **Unbounded Arrays** besonders problematisch (Bspw. Kommentare)
- Duplikation: Informationen über den Autor werden in jedem Blogpost gespeichert, den er hochlädt
→ Autorinformationen ändern kann teuer werden
→ Potential für Inkonsistenzen

Embedding umdrehen (Access Pattern Driven)

Idee

Embedding kann in beide Richtungen erfolgen
Struktur richtet sich nach häufigsten Zugriffen

Beispiel

Oft: „alle Artikel eines Autors anzeigen“
→ Autor als Top-Level-Dokument

Achtung!

Gefährlich bei vielen Artikeln (Unbounded Array)

```
{
  "_id": "author_42",
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "bio": "Tech writer."
  "articles": [
    {
      "title": "Why Document Stores Matter",
      "slug": "why-document-stores-matter",
      "content": "Document stores are useful when...",
      ...
    },
    {
      "title": "Title2",
      "content": "...",
      ...
    }
  ]
}
```

Referencing Beispiel: Website mit Blogposts

Autor-Kollektion

```
{  
  "_id": "author_42",  
  "name": "Alice Johnson",  
  "email": "alice@example.com",  
  "bio": "Tech writer."  
}
```



Artikel-Kollektion

```
{  
  "_id": "article_1001",  
  "title": "Why Document Stores Matter",  
  "slug": "why-document-stores-matter",  
  "content": "Document stores are useful when data is semi-  
structured and evolves over time...",  
  "status": "published",  
  "publishedAt": "2026-04-23T10:00:00Z",  
  "tagIds": ["tag_1", "tag_2", "tag_3"],  
  "categoryId": "cat_10",  
  "authorId": "author_42"  
}
```

Referencing Beispiel: Website mit Blogposts

PROS

- Keine Duplikation der Autoreninfo
 - Einfache updates, keine Inkonsistenzen
- Kleineres Artikel-Dokument
 - Einfachere/schnellere Lesezugriffe, **wenn wir Autoreninfo nicht brauchen**
- gut für häufig wiederverwendete Daten, z. B. Autoren, Kategorien, Tags

CONS

- Collections brauchen JOIN, wenn Autoreninfo benötigt wird
 - Langsam
- Hinzufügen eines neuen Artikels mit einem neuen Autor benötigt mehrere Schreiboperationen
- dangling references möglich, wenn referenzierte Dokumente gelöscht werden

Entscheidungskriterien

Embedding benutzen bei...

- Daten werden immer zusammen gelesen
- Beziehung ist one-to-one oder one-to-few
- Daten sind klein und begrenzt
- Daten ändern sich selten unabhängig voneinander

Referencing benutzen bei...

- Daten werden auch unabhängig genutzt
- Beziehung ist one-to-many oder many-to-many
- Daten können stark wachsen
- Du willst Redundanz vermeiden

Bei Embedding vs. Referencing immer „Access Pattern First“: Die Wahl wird aus Lese- und Schreibmustern abgeleitet.

04

Abfragemodell

15 Minuten

Wie fragt man Dokumente ab? MQL statt SQL

Selbe Idee, andere Form: der Filter wird selbst zu einem Dokument.



Aufgabe: Alle Bücher unter 40 €

SQL *(relational)*

```
SELECT * FROM produkte
WHERE kategorie = 'buch'
      AND preis < 40;
```

MQL *(MongoDB Query Language)*

```
db.produkte.find({
  "kategorie" : "buch",
  "preis" : {"$lt": 40 }
})
```

Das elegante Prinzip dahinter: Filter werden mit derselben JSON-Syntax beschrieben wie die Daten selbst.
Es gibt keine separate Query-Sprache zu lernen - nur JSON-Objekte mit speziellen Operator-Schlüsseln

CRUD - die vier Grundoperationen

Create, Read, Update, Delete - gleiche Operationen wie SQL, neue Syntax (Sprachenabhängig)

Create *neues Dokument einfügen*

```
db.produkte.insert_one({  
  "kategorie": "buch",  
  "titel": "Refactoring",  
  "preis": 42.00  
})
```

Read *Dokumente nach Filter abrufen*

```
db.produkte.find({  
  "kategorie": "album",  
  "künstler": "Beatles"  
})  
// auch findOne, find().limit(N)
```

Update *Felder ändern (Filter + \$set)*

```
db.produkte.update_one(  
  {"_id": "p_007"}, // Filter  
  {"$set": {"preis": 35.00}}  
)
```

Delete *Dokumente entfernen*

```
db.produkte.delete_one({  
  "_id": "p_018"  
})  
// auch deleteMany({...})
```

Filter- & Vergleichsoperatoren

Mehr als nur Gleichheit - Bedingungen werden als Operator-Objekte ausgedrückt.

OPERATOREN

<code>\$eq</code>	gleich
<code>\$ne</code>	ungleich
<code>\$gt</code> <code>\$gte</code>	größer / $\geq$
<code>\$lt</code> <code>\$lte</code>	kleiner / $\leq$
<code>\$in</code>	Wert in Menge (Kategorie)
<code>\$nin</code>	Wert NICHT in Menge (Kategorie)
<code>\$and</code> <code>\$or</code>	logisches UND / ODER

BEDINGUNGEN KOMBINIEREN

Alle Bücher zwischen 20 € und 50 € - *mit AND*

```
db.produkte.find({
  "kategorie": "buch",
  "preis": {
    "$gte": 20, "$lte": 50
  }
})
```

Mehrere Felder im selben Objekt = implizites AND.

Für OR: `{ $or: [ {...}, {...} ] }`

Aufgabe - schreibt die Query

Übung: von der Umgangssprache zur MQL.

AUFGABE

"Gebt mir alle Alben, die NICHT von den Beatles sind und mindestens 15 Tracks haben."

Welche Collection? Welche Felder? Welche Operatoren?

LÖSUNG

```
db.produkte.find({  
  "kategorie": "album",  
  "künstler": { "$ne": "Beatles" },  
  "tracks": { "$gte": 15 }  
})
```

\$ne für „nicht gleich“, \$gte für „mindestens“ - beide Bedingungen gelten gleichzeitig.

Dot-Notation - Zugriff auf verschachtelte Felder

Innerhalb eines Dokuments kann man mit Punkten zum Zielfeld navigieren.

Dokument

```
{
  "_id": "p_007",
  "kategorie": "buch",
  "titel": "Effective Java",
  "autor": {
    "name": "Joshua Bloch",
    "land": "USA"
  },
  "lager": [
    {"ort": "Berlin"},
    {"ort": "Hamburg"}
  ]
}
```

Zugriff per Dot-Notation

1. Verschachteltes Objekt - *Autor aus USA*

```
{ "autor.land": "USA" }
```

2. Array von Objekten - *Lager in Berlin*

```
{ "lager.ort": "Berlin" }
```

3. Array-Element - *Tag 'java' vorhanden*

```
{ "tags": "java" }
```

Die DB sieht IN das Dokument hinein - kein JOIN, keine Pfad-Tricks.

Aufgabe - verschachtelt abfragen

Jetzt die Dot-Notation anwenden.

A U F G A B E

"Findet alle Bestellungen von Kunde u_42, bei denen mindestens ein Item mehr als 50 € kostet."

*Denkt an die **Embedded-Variante**: items ist ein Array von Objekten*

L Ö S U N G

```
db.bestellungen.find({  
  "kunde": "u_42",  
  "items.preis": { "$gt": 50 }  
})
```

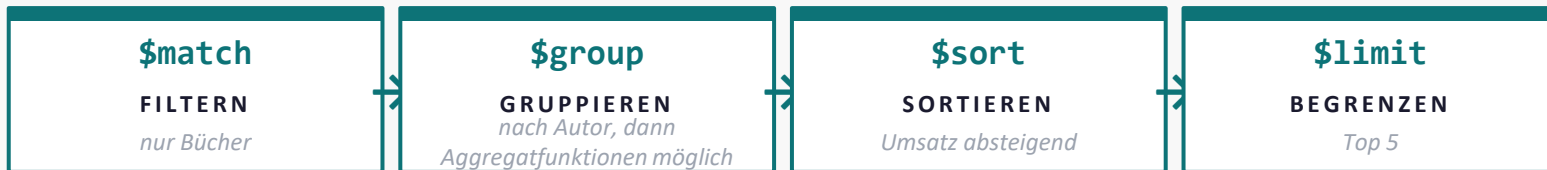
"items.preis" prüft automatisch jedes Element des Arrays - reicht, wenn EINES die Bedingung erfüllt.

Aggregation Pipeline - wenn find() nicht reicht

Find() ist ideal für einfach Filter-Abfragen. Aber für komplexe Analysen (Gruppierungen, Summen, Durchschnitte, ...) braucht man etwas komplexeres: Aggregation Pipelines

Grundprinzip

Dokumente fließen durch geordnete Kette von Stages. Jede Kette: nimmt Ausgabe von vorherigem, transformiert, reicht das Ergebnis weiter



BEISPIEL - Top 5 Autoren nach Gesamtumsatz

```
db.produkte.aggregate([
  {"$match": {"kategorie": "buch" } },
  {"$group": {"_id": "$autor.name", "umsatz": {"$sum": "$preis" } } },
  {"$sort": {"umsatz": -1 } },
  {"$limit": 5 }
])
```

Volltextsuche - Text-Indizes & Atlas Search

Wie findet man Inhalte in Textspalten, schnell, treffsicher und mit Relevanz-Ranking?

SQL LIKE

naiv, langsam, kein Ranking

- Zeichenweiser Substring-Match: findet „java“ auch mitten im Wort
- Kein Ranking: Alle Treffer gleichwertig - kein Score
- Kein Scoring: „Effective Java“ = Kommentar mit „java“ - gleiche Relevanz
- Full-Table-Scan: % am Anfang verhindert jeden Index - sehr langsam bei Millionen Zeilen

```
WHERE titel LIKE '%java%'
```

Text-Index

MQL \$text

- Wortbasiert: „java programming“ findet Dokumente mit beiden Begriffen - Reihenfolge egal
- Stoppwörter ignoriert: „the“, „und“, „der“ werden automatisch herausgefiltert
- Term-Frequency-Scoring: Häufigere Treffer = höherer Score

```
db.produkte.find({
  "$text": {"$search": "java" }
})
```

Atlas Search

Apache Lucene

- Fuzzy-Matching: „Effektiv Java“ (Tippfehler) findet trotzdem „Effective Java“
- Synonyme: Suche nach „Automobil“ findet auch Ergebnisse für „Auto“

```
db.produkte.aggregate([
  {"$search": {
    "text": {"query": "java", "fuzzy": {}}
  }
}]
```

Somit gibt es also die Möglichkeit „von Pattern-Match zur semantischen Suche“, ohne die DB zu wechseln.

05

Architekturüberblick

15 Minuten

Wie skaliert ein Document Store?

Single Node

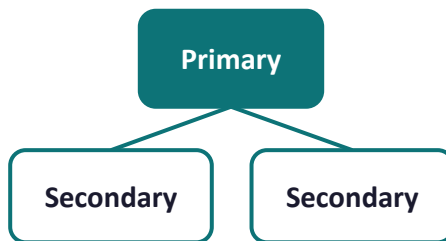
- einfach zu betreiben
- ein einzelner Knoten



Single Point of Failure

Replica Set

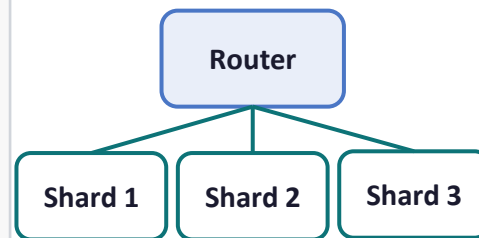
- Mehrere Kopien derselben Daten
- automatisches Failover



Ausfallsicherheit

Sharded Cluster

- Verteilt Daten über mehrere Maschinen (Shards)
- Router leitet Queries weiter

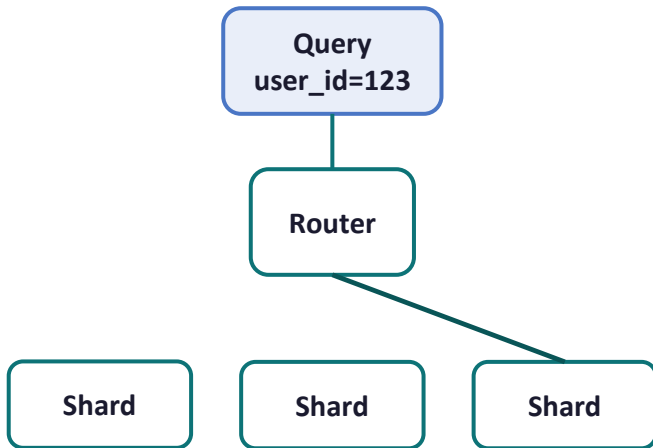


Datenmenge & Durchsatz

Query Routing: targeted vs. scatter-gather

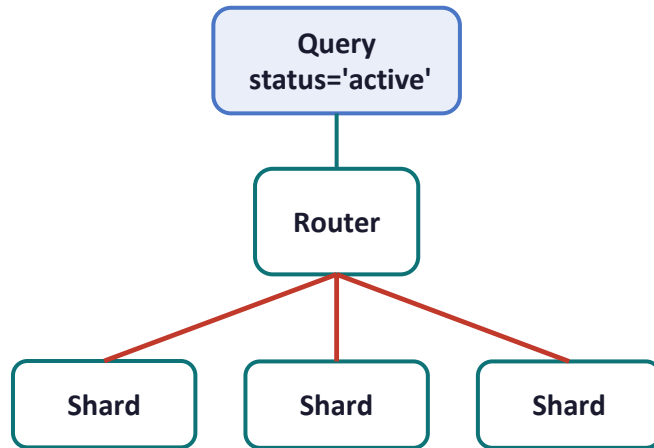
Ob eine Anfrage billig oder teuer ist, hängt oft davon ab, ob sie den Shard Key enthält.

Targeted Query



Nur relevanter Shard

Scatter-Gather Query



Alle Shards müssen antworten

Shard Keys

Der Shard Key entscheidet:

- wo ein Dokument gespeichert wird
- wie gleichmäßig Last verteilt wird
- ob Queries gezielt geroutet werden können

Beispiel Shard Key: Alter der Autoren

	Shard 1 Alter 0-24	Shard 2 Alter 25-49	Shard 3 Alter 50-74	Shard 4 Alter 75+
Datenanzahl	mittel	sehr voll	mittel	sehr leer
Lastverteilung	Hängt von den Queries ab. Shard 2 wird aber vermutlich Hotspot.			
Sinnvoll?	Nein. Typische Blogposts Queries filtern nicht nach Autoren-Alter.			

Shard Keys

Der Shard Key entscheidet:

- wo ein Dokument gespeichert wird
- wie gleichmäßig Last verteilt wird
- ob Queries gezielt geroutet werden können

Guter Shard Key

- hohe Kardinalität
- gleichmäßige Verteilung
- häufigen in Queries enthalten
- vermeidet Hotspots

Bsp.: hash(authorId)

Schlechter Shard Key

- wenige Werte, z. B. *status*
- monotone Werte, z. B. *createdAt*
- selten gefilterte Felder
- führt zu Hotspots oder Broadcast-Abfragen

Der Shard Key entscheidet, wie gut Datenverteilung und Query-Performance später skalieren

Indizes: Performance vs. Schreibaufwand

Indizes beschleunigen Reads, kosten aber Speicher, RAM und Schreibaufwand

Was bringt ein Index?

- Dokumente schnell finden, ohne die gesamte Collection zu durchsuchen
- Speichert Feldwert → Verweis auf Dokument

→ Schnelle Reads

Nachteile

- Mehr Speicher- & Ram-Verbrauch
- Jedes Insert & Update muss Indexstruktur aktualisieren

→ Langsamere Writes

Document Store Besonderheiten

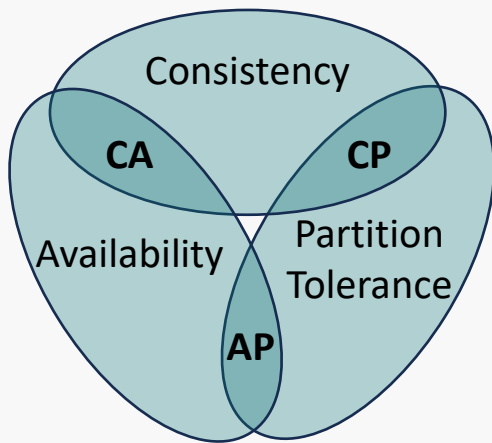
- Nested Felder & Arrays können auch indiziert werden!

- Können Indexe „aufblasen“:

```
tags: ["db", "nosql", "cloud"]
```

Arrays → viele Index-Einträge

CAP-Theorem-Einordnung



CAP

1. Consistency

alle Clients sehen denselben aktuellen Stand

2. Availability

jede Anfrage erhält eine erfolgreiche Antwort

3. Partition Tolerance

System funktioniert trotz Netzwerkfehlern weiter

Partition Tolerance ist bei verteilten Systemen nicht optional

Einordnung für Document Stores

- Document Stores sind verteilbar und daher auf Partition Tolerance ausgelegt
- Trade-off: Konsistenz vs. Verfügbarkeit
- Konfigurierbar (Read- & Write-Concern, Replikation)

Atomarität auf Dokumentenebene

Ein Dokument / Embedded

Items leben im Bestell-Dokument

```
{
  "_id": "b_889",
  "kunde": "u_42",
  "items": [
    { "id": "p_007", "preis": 39.90 },
    { "id": "p_012", "preis": 34.90 }
  ],
  "summe": 74.80 ← Atomar
}
```

Bestellung kann atomar verändert werden

Mehrere Dokumente / Referenced

*Items leben in eigener Collection, Dokument speichert
nur die Referenz*

```
{
  "_id": "b_889",
  "kunde": "u_42",
  "item_ids": [
    "p_007",
    "p_012"
  ],
  "summe": 74.80 ← Kann inkonsistent werden
}
```

Keine automatische Atomarität

Dokumentgrenze ist wichtige Designentscheidung!

Open-Source- & Cloud-Alternativen

MongoDB



- Marktführer
- Community Server (self hosted) / Atlas (managed)
- großes Ökosystem
- SSPL (keine echte Open-Source-Lizenz)

FerretDB



- Open-Source-Alternative
- PostgreSQL als Backend
- Unterstützt MongoDB-ähnliche Treiber, Tools und Query-Syntax

Apache CouchDB



- Apache Open Source
- HTTP/REST API
- stark bei Replikation
- offline-/sync-freundlich

Couchbase



- Distributed Document DB
- Community & Enterprise
- Search / Analytics / Edge
- Plattform-Ansatz

Cloud Angebote

- MongoDB Atlas
- Amazon DocumentDB
- Cloud Firestore

06

Vergleich mit relationalen Datenbanken

10 Minuten

Kurzvergleich: SQL vs. Document Store

RELATIONAL		DOCUMENT STORE
Tabellen, Zeilen, Spalten	→	Collections, Dokumente, Felder
Flache Struktur über mehrere Tabellen	→	JSON-/BSON-Dokumente mit Hierarchien
Festes Schema	→	Schema-flexibel mit optionaler Validierung
Normalisierung & JOINS	→	Embedded oder referenziert
Stark bei mehrtabelligen ACID-Transaktionen	→	Atomar vor allem auf Dokumentenebene
Klassisch eher vertikal skaliert	→	Horizontales Skalieren über Sharding

Wann SQL, wann Document Store?

SQL wählen, wenn ...

- Datenmodell stabil und stark strukturiert ist
- Integrität, Constraints und Transaktionen zentral sind
- Beziehungen häufig über viele Entitäten laufen
- viele flexible Ad-hoc-Analysen gebraucht werden
- Daten oft aus mehreren Tabellen kombiniert werden müssen

Document Store wählen, wenn ...

- Daten semi-strukturiert oder variabel sind
- Objekte natürlich hierarchisch aufgebaut sind
- Daten häufig als Ganzes gelesen/geschrieben werden
- Schema sich schnell weiterentwickelt
- horizontale Skalierung und hohe Schreib-/Leselast wichtig werden

Access Pattern first

Wie sehen die Daten aus?

Stabil und Tabellarisch
→ SQL

Variabel & Hierarchisch
→ Document Store

Wie werden Daten gelesen?

viele Kombinationen über Entitäten
→ SQL

ganze Objekte auf einmal
→ Document Store

Wie werden Daten geändert?

mehrere Entitäten müssen
gemeinsam konsistent sein
→ SQL

Änderungen bleiben meist
innerhalb eines Dokuments
→ Document Store

Typische Beispiele

RELATIONAL	DOCUMENT STORE
Banking / Zahlungen <i>starke ACID-Transaktionen, Konsistenz</i>	Produktkatalog <i>heterogene Attribute je Kategorie</i>
Buchhaltung / ERP <i>klare Entitäten, Constraints, Integrität</i>	Webseiten <i>Seiten, Content-Blöcke, Medien, Tags</i>
Reporting / Controlling <i>viele JOINS</i>	User Profile <i>viele optionale (und wachsende) Felder</i>
	Support Tickets <i>Kommentare, Anhänge, Status, Metadaten</i>
→ Klassische Unternehmenssysteme	→ Flexible, objektzentrierte Anwendungen

Kurze Pause

Bis in
10 Minuten!



07

Praktische Demo

30 Minuten

Decathlon Produktkatalog Digitalisierung



08

Challenge

20 Minuten

Challenge



GOOD LUCK

09

Kahoot Quiz

10 Minuten

10

Diskussion

10 Minuten

Diskussion

Mögliche Orientierungsfragen/Diskussionsideen:



Wie intuitiv war MQL für euch im Vergleich zu SQL?

Was hat euch am Datenmodell überrascht?

Wie seht ihr Schema-Flexibilität: hilfreiches Feature oder Gefahr?